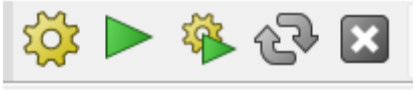
cobalt-heron-8b24 PA2 report

I. 編譯執行

點擊 CodeBlock 中的 Build and Run，即可開始編譯和執行。



cobalt-heron-8b24_PA2

```
Welcome to Knight's Treasure Hunt!!
Enter board size N, step limit, and starting energy (e.g., 7 25 5):
```

II. 完成度

本次作業完成 Basic Part，未實作 Bonus 部分。

實作內容包含以下功能，均已完整實現：

1. 能量、分數、炸彈、路障等格子行為正確實作。
2. 步數上限、能量耗損與加成完整實作。
3. DFS 搜尋 +:剪枝使用 Upper Bound 避免無效分支。使用可用步數、剩餘 Bonus 數量等要素提升剪枝效果。
4. 合法 Knight moves 的順序選擇。
5. 完整紀錄最佳路徑與修正。
6. 輸出結果格式與 Public Case 格式一致

III. Basic part 流程

1. 讀取輸入

   顯示歡迎訊息後，初始化以及輸入各項資訊。

   包含棋盤大小 N、步數限制、初始能量，以及 Bonus(B)、Energy(E)、Bomb(X) 之設定。最後讀取整個 NxN 棋盤，記錄 S、Obstacle、Bonus、Energy、Bomb。

```cpp
int main()
{
    cout << "Welcome to Knight's Treasure Hunt!!" << endl;
    cout << "Enter board size N, step limit, and starting energy (e.g., 7 25 5): ";
    cin >> board_size >> step_limit >> start_energy;
    cout << "Enter values for Bonus (B), Energy (E), and Bomb penalty (X) (e.g., 5 4 10): ";
    cin >> bonus_value >> energy_value >> bomb_penalty;
    cout << "Enter the board with " << board_size << " lines, each with " << board_size << " tokens separated by spaces: "
```

2. DFS 探索

   從起點開始遞迴搜尋所有可能的騎士走法。

   每個節點更新最佳分數與最佳路徑，並使用上界進行剪枝，無望超越最佳分數則直接停止。

3. 候選節點生成與排序

檢查 Knight 的八個移動方向全部檢查是否合法，並給定優先順序。

```
while(true)
{
    // (1) read command
    int cmd = readCommand(d);
    if (cmd==-999) return 0;
```

若在跳躍中再次輸入「跳」，會提示重新輸入。

4. 上界估計

根據剩餘可走步數、剩餘能源格、剩餘 bonus 格計算可能上限，若無法超越則提前剪枝。

5. 輸出結果

程式會根據先前的結果執行對應動作。

IV. Basic part 實作部分

1. 基本設定

```
#include <bits/stdc++.h>
using namespace std;

// ===== Knight's 8 possible directions =====
const int MOVE_COUNT = 8;
const int KNIGHT_DR[MOVE_COUNT] = {2, 1, -1, -2, -2, -1, 1, 2};
const int KNIGHT_DC[MOVE_COUNT] = {1, 2, 2, 1, -1, -2, -2, -1};

// ===== Global parameters =====
int board_size, step_limit, start_energy;
int bonus_value, energy_value, bomb_penalty;

// ===== Board attributes =====
vector<vector<bool>> wall_cell;
vector<vector<bool>> bonus_cell;
vector<vector<bool>> energy_cell;
vector<vector<bool>> bomb_cell;
int start_row, start_col;

// ===== Search state =====
vector<vector<bool>> visited_cell;
vector<int> best_path_row, best_path_col;
vector<int> current_path_row, current_path_col;
int remaining_energy = 0, remaining_bonus = 0, remaining_walkable = 0;
int best_total_score = -1;
long long total_visited_nodes = 0;
```

在程式的開頭，我們先定義騎士的移動方向、棋盤參數以及各類格子的狀態。此區段主要進行資料結構初始化，讓後續的 DFS 搜尋能正確判斷棋盤狀態。

```cpp
void setup_board()
{
    wall_cell.assign(board_size, vector<bool>(board_size, false));
    bonus_cell.assign(board_size, vector<bool>(board_size, false));
    energy_cell.assign(board_size, vector<bool>(board_size, false));
    bomb_cell.assign(board_size, vector<bool>(board_size, false));
    visited_cell.assign(board_size, vector<bool>(board_size, false));
    remaining_walkable = board_size * board_size;

    for (int r = 0; r < board_size; r++)
        for (int c = 0; c < board_size; c++)
        {
            char token;
            cin >> token;
            if (token == 'S')
            {
                start_row = r;
                start_col = c;
                remaining_walkable--;
            }
            if (token == '#')
            {
                wall_cell[r][c] = true;
                remaining_walkable--;
            }
            if (token == 'B')
            {
                bonus_cell[r][c] = true;
                remaining_bonus++;
            }
            if (token == 'E')
            {
                energy_cell[r][c] = true;
                remaining_energy++;
            }
            if (token == 'X')
                bomb_cell[r][c] = true;
        }

    current_path_row.push_back(start_row);
    current_path_col.push_back(start_col);
    remaining_walkable--;
    visited_cell[start_row][start_col] = true;
}
```

接著做初始化。

2. DFS

DFS 從起點開始，不斷往所有可能的騎士步伐展開，並在抵達限制條件（步數上限或能量耗盡）前，持續累積分數並記錄當前路徑。

為了避免不必要的暴力搜尋，我在 DFS 開始的第一行加入剪枝。若目前分數加上未來最多可取得的理論分數，仍舊無法超越目前的最佳解，則直接停止此分支的探查，以提升整體效率。

```cpp
// ===== Depth-first search with pruning =====
void search_knight(int r, int c, int step, int energy, int score)
{
    ++total_visited_nodes;
    if (upper_bound_estimate(step, energy, score) <= best_total_score)
        return;

    if (score > best_total_score)
    {
        best_total_score = score;
        best_path_row = current_path_row;
        best_path_col = current_path_col;
    }

    if (step >= step_limit || energy <= 0)
        return;

    int move_r[8], move_c[8], move_p[8];
    int move_count = 0;
    generate_moves(r, c, move_r, move_c, move_p, move_count);

    for (int i = 0; i < move_count; i++)
    {
        int nr = move_r[i];
        int nc = move_c[i];
        int next_energy = energy - 1 + (energy_cell[nr][nc] ? energy_value : 0);
        int next_score = score + (bonus_cell[nr][nc] ? bonus_value : 0) - (bomb_cell[nr][nc] ? bomb_penalty : 0) + 1;

        visited_cell[nr][nc] = true;
        current_path_row.push_back(nr);
        current_path_col.push_back(nc);

        if (energy_cell[nr][nc])
            remaining_energy--;
        else if (bonus_cell[nr][nc])
            remaining_bonus--;
        remaining_walkable--;

        search_knight(nr, nc, step + 1, next_energy, next_score);

        if (energy_cell[nr][nc])
            remaining_energy++;
        else if (bonus_cell[nr][nc])
            remaining_bonus++;
        remaining_walkable++;
        current_path_row.pop_back();
        current_path_col.pop_back();
        visited_cell[nr][nc] = false;
    }
```

3. 下一步選擇

```cpp
// ===== Generate possible knight moves =====
void generate_moves(int r, int c, int move_r[], int move_c[], int move_priority[], int &count)
{
    count = 0;
    for (int k = 0; k < MOVE_COUNT; k++)
    {
        int nr = r + KNIGHT_DR[k], nc = c + KNIGHT_DC[k];
        if (!inside_board(nr, nc))
            continue;
        if (wall_cell[nr][nc])
            continue;
        if (visited_cell[nr][nc])
            continue;

        int add_score = 1;
        if (bonus_cell[nr][nc])
            add_score += bonus_value;
        else if (bomb_cell[nr][nc])
            add_score -= bomb_penalty;

        int add_energy = energy_cell[nr][nc] ? energy_value : 0;

        int degree = 0;
        for (int d = 0; d < MOVE_COUNT; d++)
        {
            int rr = nr + KNIGHT_DR[d], cc = nc + KNIGHT_DC[d];
            if (inside_board(rr, cc) && !wall_cell[rr][cc])
                degree++;
        }

        int priority = 4 * add_score + 2 * add_energy - degree;
        move_r[count] = nr;
        move_c[count] = nc;
        move_priority[count] = priority;
        count++;
    }

    // Manual insertion sort
    for (int i = 1; i < count; i++)
    {
        int temp_r = move_r[i], temp_c = move_c[i], temp_p = move_priority[i];
        int j = i - 1;
        while (j >= 0 && (move_priority[j] < temp_p || (move_priority[j] == temp_p && (move_r[j] * board_size + move_c[j]) > (temp_r * board_size
        {
            move_r[j + 1] = move_r[j];
            move_c[j + 1] = move_c[j];
            move_priority[j + 1] = move_priority[j];
            j--;
        }
        move_r[j + 1] = temp_r;
        move_c[j + 1] = temp_c;
        move_priority[j + 1] = temp_p;
    }
}
```

在每一次遞迴展開下一層 DFS 前，我會先產生所有合法的騎士移動，並為每個候選格子計算一個 priority，再利用 insertion sort 進行排序，使 DFS 盡量先走向更有希望得到高分的方向。這樣可以有效提升剪枝成功率，加快整體搜尋速度。
每個候選格子的 priority 由以下三部分組成：

1. score gain：
   若下一格是 Bonus 則大幅加分；若是 Bomb 則會扣分；一般格子則固定 +1 分。
2. energy gain：
   若下一格是 Energy 則會額外補充能量，使未來能走更多步，給予較高的比重。
3. Warnsdorff-like degree：
   計算從下一格還能走到多少格，degree 愈高優先度愈低（負向加權）。
4. 經過上述三項資訊組合後，我為每格計算
   priority = 4 × score + 2 × energy – degree

   最後使用 Insertion Sort 將候選步依照 priority 由大到小排列。

```
/* ===== Rendering (fixed width per cell) ===== */
static void draw(const int world[COLS], const Dino& d)
{
    const bool RIGHT_ALIGN = true;

    for (int r=1; r<=ROWS; ++r)
    {
        for (int c=0; c<COLS; ++c)
        {
            char ch = ' ';
            bool dHere = (c==0 && dinoAtRow(d,r));
            bool oHere = obsAtRow(world[c], r);

            if (dHere) ch='D';
            else if (oHere) ch = (world[c]==C1 || world[c]==C2) ? 'C' : 'P';

            cout << '|';
            if (RIGHT_ALIGN) cout << ' ' << ch;
            else cout << ch << ' ';
        }
        cout << "|\n";
    }
}
```

4. 輸出結果
   首先印出搜尋的最終結果，包括最佳得分和總步數。

```
----The Result Of Knight's Treasure Hunt----
Best score: 4
Steps: 3
```

接著重播移動的路徑。

```
----The Replay Of Knight's Treasure Hunt----
Step  (x,y)     Type        Score     Energy
-----------------------------------------------
0     (0,3)     start       0         4
1     (2,2)     bomb        0         3
2     (0,1)     normal      1         2
3     (2,0)     bonus       4         1
```

```cpp
// ===== Show final board =====
void show_board()
{
    vector<vector<string>> board_print(board_size, vector<string>(board_size, "0"));
    for (int r = 0; r < board_size; r++)
        for (int c = 0; c < board_size; c++)
        {
            if (wall_cell[r][c])
                board_print[r][c] = "#";
            else if (bonus_cell[r][c])
                board_print[r][c] = "B";
            else if (energy_cell[r][c])
                board_print[r][c] = "E";
            else if (bomb_cell[r][c])
                board_print[r][c] = "X";
        }

    for (size_t i = 0; i < best_path_row.size(); i++)
    {
        int r = best_path_row[i], c = best_path_col[i];
        board_print[r][c] = (i == 0 ? "S" : to_string(i));
    }

    int maxStep = best_path_row.size() - 1, width = 2;
    while (maxStep >= 10)
    {
        width++;
        maxStep /= 10;
    }

    cout << "----The Best Path Grid Of Knight's Treasure Hunt----" << endl;
    for (int r = 0; r < board_size; r++)
    {
        for (int c = 0; c < board_size; c++)
            cout << setw(width + 1) << board_print[r][c];
        cout << endl;
    }
}

// ===== Show summary =====
void show_summary()
{
    cout << "\n----The Result Of Knight's Treasure Hunt----" << endl;
    cout << "Best score: " << best_total_score << endl;
    cout << "Steps: " << best_path_row.size() - 1 << endl;
}
```

最後顯示標示完整最佳路徑的棋盤。

```
----The Best Path Grid Of Knight's Treasure Hunt----
S  0  0  0
0  X  1  #
2  X  0  0
0  0  3  0
```

```cpp
// ===== Replay the best path =====
void show_replay()
{
    cout << "\n----The Replay Of Knight's Treasure Hunt----" << endl;
    cout << left
         << setw(6) << "Step"
         << setw(10) << "(x,y)"
         << setw(12) << "Type"
         << setw(10) << "Score"
         << setw(10) << "Energy" << endl;
    cout << string(48, '-') << endl;

    int score = 0, energy = start_energy;
    for (size_t i = 0; i < best_path_row.size(); ++i)
    {
        int r = best_path_row[i], c = best_path_col[i];
        string cell_type = "normal";
        if (i == 0)
            cell_type = "start";
        else
        {
            ++score;
            --energy;
            if (bonus_cell[r][c])
            {
                cell_type = "bonus";
                score += bonus_value;
            }
            else if (energy_cell[r][c])
            {
                cell_type = "energy";
                energy += energy_value;
            }
            else if (bomb_cell[r][c])
            {
                cell_type = "bomb";
                score -= bomb_penalty;
            }
        }
        int x = c, y = board_size - 1 - r;
        cout << left << setw(6) << (int)i
             << setw(10) << ("(" + to_string(x) + "," + to_string(y) + ")")
             << setw(12) << cell_type
             << setw(10) << score
             << setw(10) << energy << endl;
    }
}
```

V. 心得

在本次作業中，我學到了如何在 DFS 中加入有效剪枝策略，尤其是 Upper Bound 的設計，能大幅減少不必要的遞迴深度。此外，由於作業限制不能使用 struct 與 class，也不能使用 STL sort，因此需要自行規劃資料結構並手刻 insertion sort。這讓我更理解資料結構底層運作方式，也提升了在受限環境下進行程式設計的能力。